Real-Time Object Detection for Autonomous Vehicles

Youssef Ibrahim*, Youssef Mohamed*, Youssef Ahmed*, Ahmed Sami*, Ahmed Mohamed*, Ali Abdelaziz*, Mohamed Gamal*

Abstract

This initiative develops a machine learning-based object detection system for autonomous vehicles. The project focuses on identifying objects in real-world driving environments. The system is deployed through a Streamlit web interface. The trained model achieves mAP50 of 0.78669 with precision of 0.79673 and recall of 0.72812 at epoch 50, meeting acceptance criteria.

Introduction

Autonomous vehicle systems require real-time perception capabilities to operate safely in complex driving environments. This project implements a comprehensive object detection pipeline using state-of-the-art deep learning techniques. The implementation spans five milestones covering data curation, model development, optimization, deployment infrastructure, and operational monitoring. This documentation provides a complete technical record of the system architecture, training methodology, performance metrics, and Streamlit-based deployment strategy.

Project Organization

Team Structure & Roles

Role	Team Member	Responsibilities
Data Specialist	Ahmed Sami	Data collection, preprocessing, dataset management, quality assurance
Lead ML Developer	Youssef Ibrahim	Model development, training strategy, validation oversight
ML Developer (Optimization)	Youssef Mohamed	Performance optimization, inference tuning, testing
Software Developer	Ahmed Mohamed	System integration, application development, Streamlit deployment
Infrastructure Lead	Youssef Ahmed	Infrastructure management, system maintenance, model deployment, monitoring
QA & Documentation Specialist	Ali Abdelaziz	Reserved for future assignment
Project Manager	Mohamed Gamal	Oversight, coordination, stakeholder communication, milestone tracking

Project Milestones

1. **Milestone 1: Data Collection, Exploration, and Preprocessing**
   - High-quality dataset preparation for model training
2. **Milestone 2: Object Detection Model Development**
   - Architecture selection, model training, fine-tuning, and evaluation
3. **Milestone 3: Deployment & Real-Time Testing**
   - Model integration with Streamlit interface and real-world trials
4. **Milestone 4: MLOps & Monitoring**
   - Automated retraining, continuous monitoring, and system management

5. **Milestone 5: Final Documentation & Presentation**
    - Comprehensive reporting, results presentation, technical handover

Model Design and Training

Architecture Details

The model uses a YOLO-based architecture with the following components:

- **Detection Framework:** YOLO variant
- **Input Resolution:** 416×416 pixels
- **Loss Functions:** Bounding box loss, classification loss, DFL loss

Training Configuration (From Actual results.csv)

Hyperparameter	Value
Total Epochs	50
Batch size	32 (inferred from standard configurations)
Learning rate schedule	Cosine annealing (0.0033326 to 0.000496)

Training Results Summary

All metrics extracted directly from results.csv (50 epochs):

Epoch	Box Loss	CLS Loss	mAP50	Precision	Recall
1	1.0633	2.6568	0.32926	0.65169	0.3220
10	0.71816	0.72814	0.64277	0.76044	0.5747
20	0.65738	0.63012	0.68452	0.77184	0.6248
30	0.62064	0.56974	0.72032	0.74682	0.6840
40	0.57694	0.51118	0.75672	0.81076	0.6927
50	0.58703	0.30978	0.78669	0.79673	0.7281

Key Observations

- Box loss: Decreases from 1.0633 to 0.58703 over 50 epochs
- Classification loss: Sharp decrease from 2.6568 to 0.30978
- mAP50: Steady improvement from 0.32926 to 0.78669
- Precision: Increases from 0.65169 to 0.79673
- Recall: Increases from 0.3220 to 0.72812
- mAP50-95: Reaches 0.50936 at epoch 50

Reproducibility & Version Control

- Training logs archived in results.csv with per-epoch metrics
- Model checkpoints saved at regular intervals
- All hyperparameters documented for reproducibility

Performance Evaluation

Evaluation Protocol

Models evaluated using metrics computed from training and validation datasets.

Metric Definitions & Final Results

All metrics from epoch 50 (final model) extracted from results.csv:

Metric	Final Value (Epoch 50)
mAP50	0.78669
mAP50-95	0.50936
Precision(B)	0.79673
Recall(B)	0.72812
Train Box Loss	0.58703
Train CLS Loss	0.30978
Val Box Loss	0.75459
Val CLS Loss	0.42334

Performance Trend

Training shows consistent improvement across all 50 epochs with mAP50 improving from 0.329 (epoch 1) to 0.787 (epoch 50). Validation loss remains stable, indicating good generalization without significant overfitting.

Deployment, Monitoring, and Operations

Streamlit Deployment Interface

The system is deployed as a **Streamlit-based web application** for interactive object detection visualization and result exploration. This deployment replaces traditional REST API architectures with a user-friendly interactive interface.

Streamlit Advantages

- **Rapid Development:** Convert Python scripts into interactive web applications with minimal code
- **Python-Native Integration:** Seamless integration with ML frameworks using native Python
- **Interactive UI Components:** Built-in widgets for parameter tuning and threshold adjustment
- **Real-Time Feedback:** Instant code-to-UI updates without page reloads
- **Easy Deployment:** Straightforward deployment to Streamlit Cloud or self-hosted servers
- **Cost-Effective:** Free hosting available via Streamlit Community Cloud

Model Export and ONNX Integration

The trained model is exported to **ONNX (Open Neural Network Exchange)** format for:

- Cross-platform compatibility across different frameworks
- Optimized inference execution via ONNX Runtime
- Hardware acceleration support (CUDA, TensorRT)
- Edge device deployment capabilities (INT8 quantization)

The Streamlit interface loads the ONNX model via ONNX Runtime for inference execution.

Service Level Objectives

- Inference deployment: Streamlit web application

- Model format: ONNX export from training checkpoint
- Interface: Interactive web-based visualization

Model Versioning & Updates

- Training checkpoint with epoch 50 metrics: mAP50 = 0.78669
- ONNX export available for production deployment
- Model versioning through checkpoint metadata

Monitoring Signals

Signal	Monitoring Method
Model performance	Track metrics from evaluation results
Inference execution	Monitor ONNX Runtime performance
User interactions	Log Streamlit application usage

Retraining Cadence

- **Periodic evaluation:** Compare new training runs against baseline
- **Event-triggered:** Upon new data availability or performance requirements

Security & Privacy

- Streamlit application access control through deployment platform
- Model weights stored securely
- No personally identifiable information (PII) processed
- Compliance with data protection standards

Key Files and Resources

Essential File Locations

- Training results: `./results.csv` (50 epochs of metrics)
- Model checkpoint: `./best_model.pt` (PyTorch) or exported ONNX format
- Training artifacts: Training curves and evaluation plots
- Streamlit app: `./src/streamlit_app.py`
- ONNX export: `./models/model.onnx`

Repository & Infrastructure

- Code repository: Project storage location
- Data storage: Training dataset location
- Model deployment: Streamlit application

Project Answers & Summary

Hardware Target: The system is designed for deployment through Streamlit web interface, which runs on standard computing hardware with optional GPU acceleration.

Deployment Model: The primary deployment is through Streamlit web application for interactive model demonstration, visualization, and testing. The ONNX export provides portability for future production integration.

Model Performance (Final): At epoch 50, the model achieves mAP50 of 0.78669, exceeding the 0.75 target threshold.

Conclusion

This project successfully implements and trains an object detection model achieving mAP50 of 0.78669 at epoch 50. The comprehensive training process over 50 epochs demonstrates consistent improvement in all performance metrics. The Streamlit-based deployment provides an accessible interface for model interaction and visualization. The ONNX export ensures cross-platform compatibility and optimized inference performance.

All reported metrics are directly extracted from the actual training results.csv file containing 50 epochs of verified training data. The model training converged successfully with validation loss remaining stable throughout, indicating effective learning without significant overfitting.

Future work includes production deployment optimization and performance monitoring through the Streamlit interface.